

Menu Name	Web Api	Table	Code Flow
Login	POST /api/Auth/Login	SECUsers secusersprofile Mastercomapany LoginLogs	• Input Validation • Check if <code>userName</code> or <code>userPass</code> is empty. • If empty → return 422 <code>BadRequest</code>. • Password Handling • Trim password. • Encrypt password using <code>EncryptDecryptPassword.Encrypt</code>. • User Lookup • If <code>LoginId</code> length = 6 → look up via <code>SECUsersProfiles</code> → then <code>Secusers</code>. • Else → look up directly in <code>Secusers</code>. • If no match → return 401 <code>Unauthorized</code>. • User Status Check • If user inactive → return 401 <code>Unauthorized</code>.

			• SQL Query Execution • Open SQL connection. • Build query depending on whether <code>EmpId == 0</code>. • Execute query → fill <code>DataSet</code>. • DataSet Validation • If multiple records → return 409 Conflict. • If no records → attempt remote login via <code>LoginDetails</code>. ◦ If remote login succeeds → re-query local DB. ◦ If still no records → return 400 <code>BadRequest</code>. ◦ If remote login fails → return 401/409 <code>BadRequest</code>. • User Object Construction • Map DB row → <code>SecuserResponse</code> object. • Populate fields: <code>UserId</code>, <code>RoleId</code>, <code>RoleName</code>, <code>REG_NUMBER</code>, <code>EmailId</code>, <code>CompanyId</code>, <code>ProfileImage</code>, <code>CompanyLogo</code>, etc. • Generate JWT token via <code>_tokenHelper.GenerateToken</code>.
--	--	--	--

			• Save login log. • Special Cases • Parent User Handling: If <code>parentuserid</code> exists, fetch parent login details. • BEMA Registration: If enabled, validate against external API. • Inactive Local User: Send mail verification. • MFA Process: If enabled, generate OTP (mail or MFA). • API Connection Failures: Return success but note "Local only". • Final Response • If all checks pass → return 200 OK with <code>Login success</code>. • Else → return appropriate error response (400, 401, 409).
DashBoard	GET <code>/api/DashBoard/load_dashboard</code>	<code>process_contributions</code> <code>PROCESS_C3Header</code> <code>onlinepayments</code> <code>Masteremployee</code>	1. Function Start • Method: <code>load_dashboard(...)</code> • Input: <code>CompanyId</code>,

			StartMonths, endMonths, StartYear, EndYear, ResultArea • Output: List<PayPeriod1> (Key-Value pairs for dashboard) 2. Date Setup IsLastmonthEMPC3Submitted IsLastmonthDirC3Submitted 3. Employee C3 Data Load • Stored Procedure: get_C3Genrated_data • Result: DataTable dt Loop over rows: • Count submissions (c3count, Pendingc3count) • Check Is_submitted → Pending or Submitted • Create C3Header object with all details (wages, contributions, penalties, notes, payment info) • Add to listDVOC3Header Contribution Calculation: • If Paid → Add to total_con • If Unpaid → Add to unPaid
--	--	--	--

			ReturnData Updates: • Total, Paid Months, Unpaid Months, Pending C3, Foreground color • Serialize DVOC3Header into JSON for dashboard list 4. Director C3 Data Load • Stored Procedure: Get_Director_C3_Genrated_data • Result: DataTable Director_dt Loop over rows: • Count submissions (D3count, Pendingc3count) • Check Is_submitted • Create C3Header object for Director • Add to Director_listDVOC3Header Contribution Calculation: • If Paid → Add to Director_total_con • If Unpaid → Add to unPaidDir ReturnData Updates: • Director Total, Paid Months,
--	--	--	--

			Unpaid Months, Pending C3, Foreground color - Serialize Director list for dashboard 5. Final Return - Total contributions - Paid/Unpaid months - Pending C3 counts - Dashboard JSON list
Employer Details	GET /api/Auth/GetAllEmployers GET /api/Auth/EmployersGetById POST /api/Auth/SaveEmployeer	Secusers, Mastercomapany secusersprofile securityquestionAnswer	- Endpoint Setup [HttpGet ("GetAllEmployers")] → The API is accessible at /GetAllEmployers. - It accepts two parameters: - CompanyId - roleid - Database Connection - Opens a SQL connection (staticConnection). - Sets audit session context for

			tracking. • Load Data • Gets all users: BLSECUsers.SECUsersCollection(). • Gets all companies: BLMasterCompany.MasterCompanyCollection(). • Filter Companies • If roleid == 1 or 2 → return all companies. • Otherwise → return only companies where: ○ CompanyId matches the input, OR ○ Parent_Id matches the input. • Count Users per Company • For each company in the filtered list: ○ Count how many users belong to that company (EmpId or SelfEmpID matches the company ID). ○ Store this count in item.UserCount.
--	--	--	---

- **Build Response**

- Close the SQL connection.
- If data exists:
 - Response includes:
 - Message = "Data retrieved successfully!"
 - Statuscode = 200
 - Status = true
 - Data = employeeerlistm (the filtered list with user counts).
- Return Ok(response).

- **Error Handling**

- If an exception occurs:
 - Log the error
(`LoggingHelper.LogError`).
 - Return a `BadRequest` with:
 - `Message = "An error occurred: ..."`
 - `Statuscode = 500`
 - `Status = false.`

=====

=====

			 SaveEmployeeer – Code Flow  Purpose This API handles Employer creation (mode=1) and update (mode=2): • mode == 1 → Add a new employer • mode == 2 → Update existing employer It also handles: • Base64 company logo upload • Registration number validation • Database updates • Optional external API registration
			1 Method Entry <pre>SaveEmployee ([FromBody] EmployeeVm employeeVm)</pre> • Input: employeeVm object with employer details and mode (1=create, 2=update).

			• Immediate check: <pre>if (employeeerVm == null) return BadRequest(new { Message = "Invalid request body", Status = false });</pre>
			2 Check if mode is 1 (Create new employer) <pre>if (employeeerVm.mode == 1) { // SQL stored procedure call to check if employer already exists }</pre> • Opens SQL connection • Calls stored procedure <pre>check_company_reg_no with: o @REG_NUMBER o @Company_Name o @OLD_REG_NUMBER o @OLD_Company_Name</pre> • Fills a DataTable • If <code>dt.Rows.Count > 0</code> → employer exists → return "Employer already registered"
			3 Initialize <code>BLMasterCompany</code> object <pre>BLMasterCompany blemployer = new BLMasterCompany();</pre>

			• Handles company logo upload if provided: 3a. Company Logo Upload • Checks if <code>employeeerVm.CompanyLogo</code> is non-empty • Splits Base64 string if it has metadata (e.g., <code>data:image/png;base64,...</code>) • Converts Base64 → <code>byte[]</code> • Saves file to wwwroot/CompanyLogo folder • Generates unique filename • Sets <code>blemployer.CompanyLogo = fileName</code> • If invalid Base64 → return 500 error • If no logo provided → <code>blemployer.CompanyLogo = null</code>
			4 Set Employer Details <pre>blemployer.Parent_Id = employeeerVm.ParentCompanyId ?? 0; blemployer.RegNumber = employeeerVm.RegNumber; blemployer.CompanyName = employeeerVm.CompanyName; blemployer.TradeName = employeeerVm.TradeName;</pre>

			<pre>blemployer.ContactPerson = employeeerVm.ContactPerson; blemployer.Email = employeeerVm.Email; blemployer.Address1 = employeeerVm.Address1; blemployer.Address2 = employeeerVm.Address2; blemployer.City = employeeerVm.City; blemployer.Zip = employeeerVm.Zip; blemployer.Country = employeeerVm.Country; blemployer.Landline = employeeerVm.Landline; blemployer.Mobile = employeeerVm.Mobile; blemployer.InsertedBy = employeeerVm.CompanyId; blemployer.InsertedOn = DateTime.Now; blemployer.IsActive = true; blemployer.IsLevyExempt = employeeerVm.IsLevyExempt;</pre> • Populates all fields in the BLMasterCompany object.
			5 Registration Validation (mode=1) • Calls ValidateRegistration(employeeerVm.RegNumber, employeeerVm.Email) • Checks if registration number

			already exists in database • If exists → return Conflict (HTTP 409) • Also double-checks using direct SQL query • If exists → sets response "Employer already registered"
			6 Save New Employer 6a. If external API registration enabled (IsBEMARegistrationEnabled==1) • Calls <code>Reg_save_at_saever(0, employeerVm)</code> • If successful: ◦ Sets <code>IsVerified = false, Tokan=""</code> ◦ Calls <code>blemployer.SaveNew()</code> ◦ Adds <code>SECUsersProfile</code> record in <code>_DbContext</code> • If API fails → return corresponding status code 6b. If external API not enabled • Sets <code>IsVerified = false, Tokan=""</code> • Parses <code>RegDate</code> if provided • Calls <code>blemployer.SaveNew()</code>

			• Adds <code>SECUsersProfile</code> • Sets response → success, message = "Employer added successfully"
			7 If mode == 2 (Update existing employer) • Updates user info in <code>Secusers</code> table • Updates security questions in <code>SecurityQuestionAnswers</code> table • Checks if new registration number already exists in database (excluding current company) • Updates employer fields in <code>BLMasterCompany</code>: ◦ Handles company logo update if new Base64 provided ◦ Updates <code>SECUsersProfile / UserSecurityQuestion Answer</code> • Handles external API registration if enabled: ◦ Calls <code>Reg_save_at_saever</code> as needed ◦ Updates <code>IsVerified</code> and

			other flags • Otherwise, just updates local database records • Sets response → success, message = "Employer Updated Successfully"
			8 Exception Handling • Any exceptions during: ◦ Base64 parsing ◦ SQL queries ◦ External API calls ◦ Database updates • Are logged via LoggingHelper • Returns HTTP 500 or BadRequest with error message
			9 Return Response • Returns Ok(response) with: ◦ Status (true/false) ◦ Message ◦ Statuscode ◦ Data (optional, e.g., BLMasterCompany object)

Employee	GET /api/C3/GetAllEmployee GET /api/C3/GetEmployeeById POST /api/C3/SaveEmployee GET /api/C3/DeleteEmployee	Mastercomapany Masteremployee Secusers, MasterEmployeeIncomes	1. Input Validation - If employeerVm (the request body) is null → return BadRequest. 2. Mode = 1 (Add Employer) 1. Check Duplicate Registration - Calls stored procedure check_company_reg_no with RegNumber and CompanyName. - If employer already exists → return response "Employer already registered". 2. Handle Company Logo - If CompanyLogo is provided in Base64: - Decode and save it into wwwroot/CompanyLogo folder. - Store filename in blemployer.CompanyLogo. - If not provided → set CompanyLogo = null. 3. Fill Employer Object - Map all fields from employeerVm (RegNumber, CompanyName,
----------	---	--	--

			ContactPerson, Email, Address, etc.) into blemployer. 4. Validate Again ○ Calls ValidateRegistration and checks if registration number/email already exists. ○ If duplicate → return Conflict. 5. Save Employer ○ If external BEMA registration is enabled: ▪ Call Reg_save_at_saver. ever. ▪ If success → save employer (SaveNew) and also add a SECUsersProfile entry. ▪ If failure → return error response. ○ If BEMA not enabled: ▪ Save employer locally (SaveNew). ▪ Add SECUsersProfile. e. ▪ Return success
--	--	--	---

			response "Employer added successfully". 3. Mode = 2 (Update Employer) 1. Update User Info Update Secusers table with new FirstName, LastName. Update SecurityQuestionAnswers with new questions/answers. 2. Check Duplicate Registration Query MasterCompany to ensure no other company has the same RegNumber. If duplicate → return "Employer already registered". 3. Update Employer Load existing company record. Update fields (UpdatedBy, UpdatedOn, etc.). If BEMA registration enabled: Call VerifyEmployerDetails and Reg_save_at_sa
--	--	--	---

			ever. ▪ If success → update employer (Update). ▪ If failure → return error response. ○ If BEMA not enabled: ▪ Handle logo update (save new image or keep old). ▪ Update employer record (Update). ▪ Update security questions (BLUserSecurityQuestionAnswer.Update()). ▪ Return success response "Employer Updated Successfully". 4. Error Handling • Any exception → log error (LoggingHelper.LogError) and return 500 Internal Server Error with message.
--	--	--	--

			===== =====  DeleteEmployee – Code Flow  Purpose This API deletes an employee if they are not associated with a C3 record. • Input parameters: ◦ EmployeeId → ID of the employee to delete ◦ isc3created → optional flag indicating if a C3 record exists for this employee
			1 Method Entry DeleteEmployee(int EmployeeId, bool? isc3created) • Creates a ResponseModel object to structure the response. ResponseModel response = new ResponseModel();
			2 Check if employee has C3 created if (isc3created == true)

			<pre>{ response.Status = false; response.Message = "You can not delete this employee as C3 exists for this employee!"; response.StatusCode = 409; return Conflict(response); }</pre> • If <code>isc3created</code> is true: ◦ The employee cannot be deleted ◦ Returns HTTP 409 Conflict with a message • If <code>isc3created</code> is null or false, proceed to deletion.
			3 Delete Employee <pre>BLMasterEmployee Employee_row = new BLMasterEmployee(); Employee_row.EmployeeID = EmployeeId; Employee_row.Delete();</pre> • Creates a new instance of <code>BLMasterEmployee</code> • Sets the <code>EmployeeID</code> • Calls <code>Delete()</code> method (assumes it handles the actual database deletion)
			4 Set Response on Success

			<pre>response.Status = true; response.Message = "Employee deleted successfully."; response.StatusCode = 200; response.Data = Employee_row;</pre> • Sets response status as success • Returns HTTP 200 OK • Includes deleted employee object in <code>Data</code> (optional, may only contain ID)
			5 Exception Handling <pre>catch (Exception ex) { var (controller, action) = ExceptionMiddleware.GetActionInfo(this); LoggingHelper.LogError(controller, action, ex.Message, ex.StackTrace); response.Message = ex.Message; response.StatusCode = 404; }</pre> • Any exception in the deletion process: ◦ Logs the error ◦ Sets response message to exception message ◦ Status code is set to 404 (not found) ◦ Returns Ok(response)

			(note: technically this returns HTTP 200, but the model inside indicates failure)
Holiday/Otherpayments	POST /api/C3/GetAllHolidayPay GET /api/C3/GetAllHolidayPayById GET /api/C3/HolidaypayDelete	MasterHolidayPayDetails MasterEmployee	1. Method Start • Input: CompanyId • Output: ResponseModel (contains message, status, status code, and data) 2. Database Connection • Get a static SQL connection from C3WizardLayerConn_BaseData . • Set audit session context. • Open connection if closed. 3. Execute Stored Procedure • Stored Procedure: Holiday_Pay_Employee_Select_All • Parameter: @CompanyId

			• Fill results into a DataTable. 4. Map Results to Objects • If rows exist in DataTable: ◦ Loop through each row. ◦ Create a Holiday_Pay_Details object. ◦ Map columns: ▪ holidayPayId, employeeID, company_ID ▪ Amount (rounded to 2 decimals) ▪ st_Date, en_Date ▪ PayNoOfTimes, PayStatus ▪ employeename, emptype ▪ IsWithoutLeave (logic: if column 10 == "True" → "Holiday Pay With Leave", else use column 11) ▪ otherHpaydes ▪ IsempDirector (icon path based on emptype) ▪ yearName (extracted from st_Date)
--	--	--	--

			○ Add object to userlist. 5. Build Response • If <code>userlist.Count > 0</code>: ○ <code>Message = "Data retrieved successfully!"</code> ○ <code>Statuscode = 200</code> ○ <code>Status = true</code> ○ <code>Data = userlist</code> • Else: ○ <code>Message = "Data Not Founds!"</code> ○ <code>Statuscode = 404</code> ○ <code>Status = false</code> ○ <code>Data = empty array</code> 6. Return • Return the <code>response</code> object.
C3 Generation	POST <code>/api/C3/LoadEmployee</code> POST <code>/api/C3/SaveNContinue</code> POST <code>/api/C3/EditC3EmployeeListing</code> GET <code>/api/C3/DeleteC3</code> GET <code>/api/C3/C3Report</code>	MasterHolidayPayDetails MasterEmployee Mastercomapany process_contributions PROCESS_C3Header BonusPayDetails	1. Method Start • Input: ○ <code>CompanyId</code> ○ <code>monthno</code> (month number) ○ <code>Year</code> ○ <code>CmbPayPeriod</code> (list of pay periods selected) ○ <code>isNilReturn</code> (flag to ignore employees)

POST
`/api/C3/SubmitC3Bulk`

- Output: `ActionResult` with employee contribution data.

2. Setup

- Calculate number of Mondays in the month → check if 5 weeks exist (`IsWeekfifth`).
- Get month start and end dates.
- Open SQL connection and set audit context.

3. Load Employees

- Get all employees for the company:
`BLMasterEmployee.MasterEmployeeCollection_grid(CompanyId)`.
- Filter employees:
 - Include only those active during the month (appointed before end date, not terminated before start date).
- If `isNilReturn == true` → clear employee list.

4. Categorize Employees by Pay Period

- Separate employees into lists:
 - Weekly (`W`)
 - Monthly (`M`)
 - Every-2-Weeks (`E2W`)

			○ Twice-Monthly (2M) ○ Or All (all categories together). 5. Holiday Pay Dates • Generate holiday pay dates for the month using <code>_C3.Holiday_Pay_Dates1</code>. 6. Process Employees (Example: Twice-Monthly) For each employee: • Create <code>C3Contributions</code> object. • Fill basic info: SSN, EmployeeId, Name, Department, Join/Terminate dates. • Mark weekly eligibility (<code>WEEK1-WEEK5</code>) based on termination date vs holiday pay dates. • Calculate holiday pay per week (<code>HPay_Week1-HPay_Week5</code>) using <code>_C3.LoadHpay</code>. • Track holiday pay ID if found. • Sum total holiday pay (<code>Ttotal_hpay</code>). • Calculate other pay and director wages pay. • Calculate wages and bonus. • Build remarks string:
--	--	--	---

			○ Holiday pay remarks ○ Other pay type ○ Director wages ○ Bonus • Set flags: IsHPAY, IsRemarkDisable, IsLevyExempt, etc. • Add to Tw_Employee_list and AllEmployee_List. 7. Process Other Pay Frequencies • Repeat similar logic for E2W, Weekly, and Monthly employees: ○ Calculate weekly holiday pay ○ Wages, bonuses, remarks ○ Add to respective lists and AllEmployee_List. 8. Final Output • After processing all employees: ○ Lists (Tw_Employee_list, E_2_W_employee_list, Weeklyemplist, Monthlyemplist, AllEmployee_List) are populated. ○ Each employee has detailed contribution info for the month.
--	--	--	--

			Return result as IActionResult. ===== ===== 1 Validate Header Exists <pre>var checkResult = await _C3.AllEmployeeExistsAsync(obj.h eaderId, false, obj.CompanyId);</pre> ✗ If failed → returns 400 BadRequest
			2 Load C3 Header List Stored Procedure → get_C3Genrated_data Output HeaderID, RegNo, PeriodMonth, Per isNilReturn... headerList = SP(get_C3Genrated_data)
			3 Find Selected Header <pre>var headerDetails = headerList.FirstOrDefault(x => x.HeaderID == obj.headerId);</pre>

			✗ If not found → 404 NotFound
			4 Compute Month / Holiday Pay Date Logic Variable monthIndex / yearValue Month & year mondays Count Monday isWeekfifth Is month havin todayDate 1st of selected holidayPayDates From _C3.Ho holidayPaydates From DB (Mas
			5 Load Employee Data Stored Procedure → get_C3Genrated_Employee_data Each row → C3Contributions F

			SSN, Name, Salary, Week flags, Wages, Bonus,
			6 Calculate Employee Salary + Holiday Pay For every employee: Calculation LoadWages() LoadBonus() LoadHpay() (Week1 → Week5) holidayPayId() fallback logic OtherPay(), OtherDirWagespay() Remarks formation
			7 Pay Frequency Switch Case PayFreq Logic W Weekly E2W Every 2 Weeks B Bi-Weekly

			MMonthly 2MTwice Monthly SSemi-Monthly This block modifies: - WAGES1 → WAGES5 - HPay_Week1 → HPay_Week5 - PayPeriod - emp_add_remove
			8Filter by Join & Terminate Date allEmployeeList = allEmployeeList .Where(x => (x.Date_terminated == null TermDate >= MonthStart) && (x.Date_Joining == null JoinDate <= MonthEnd)) .ToList();
			9Nil Return Logic Case Header not Nil + popup null → remove C3HE Header Nil → return empty list

			isNilReturn true → return empty isNilReturn false → remove non-header employ
			[10] Final Response <pre>return Ok(new ResponseModel { Status = true, Message = "Data fetched successfully.", Data = responseData });</pre>
Bonus	GET /api/C3/Load_Wages_Bonus_PayEmployee POST /api/C3/UpdateBonus	WagesPayDetails MasterEmployee BonusPayDetails	1. Input Validation • If ModelState is invalid → return BadRequest. 2. Setup Variables • Extract employee ID and SSN from request.EMPLOYEESSN. • Get employee list for the company. • Filter employees by SSN and check if they are active during the pay date (not terminated before, appointed before). • Calculate month number, year, days in month, and start date (Today_date).

			3. Check Employee Active - If employee exists in filtered list: - Assume <code>isfinalize = true</code> (C3 finalize check). - Prepare holiday pay date list (<code>days_list</code>) with one entry for the bonus pay date and amount. 4. Database Update - Open SQL connection. - Call stored procedure: <code>Bonus_Pay_Update</code> with parameters: <code>PayId</code>, <code>From_Date</code>, <code>To_Date</code>, <code>Amount</code>, <code>EmployeeId</code>, <code>CompanyId</code>, <code>PayNoOfTimes</code>, <code>EmployeeDetails</code>, <code>insertedon</code>, <code>updateon</code>, <code>PERIODD_MONTH</code>, <code>PERIOD_YEAR</code>, <code>Wages_Bouns_PayDate</code>. - Fill results into <code>DataTable</code>. 5. Response Based on Result - If <code>DataTable.Rows.Count > 0</code>: - Return success response: <code>"Bonus Update"</code>
--	--	--	---

			successfully." • Else: ◦ Return response: "Record already exist for this month!" 6. If Employee Not Active • If employee is terminated before pay date → return message: "You can't pay bonus to this employee because this employee is terminated on [date]." • If employee is appointed after pay date → return message: "You can't pay bonus to this employee because this employee is appointed on [date]." 7. Error Handling • If exception occurs: ◦ Log error (LoggingHelper.LogError). ◦ Return 500 Internal Server Error with message. 8. Default Return • If none of the above conditions match → return 404 Not Found
--	--	--	--

			with "Internal server error".
Reports	GET /api/DashBoard/load_dashboard GET /api/C3/btn_DownloadSubmittedC3_Click GET /api/C3/btn_ImportLastC3_Click	process_contributions PROCESS_C3Header onlinepayments Masteremployee	1. Method Entry & Initial Checks - Method is an async MVC controller action. - Creates a ResponseModel to return status and messages. Checks connectivity: - ConnectionInfom.HasConnection() - ConnectionInfom.APICConnection(configuration) - If either fails → returns error response.
			2. Date Range Preparation - Converts input Year and endYear to integers. - Builds: - startDate = first day of FromMonth - endDate = last day of ToMonth - Formats dates as dd-MM-yyyy. - Example:

			• <code>statedate = 01-01-2025</code> • <code>enddate = 31-03-2025</code>
			3. Build C3 Submitted API URL • Gets employer registration number: • <code>regn = _C3.regno(Company)</code> • Builds API endpoint: • <code>/C3/{regn}/C3Submitted/ER/range/{statedate}/{enddate},EE</code> • Creates <code>HttpClient</code> with: ◦ <code>Timeout = 2 minutes</code> ◦ Basic Authentication ▪ Either BEMA auth or local user credentials
			4. Call C3 Submitted Header API • Executes <code>GET</code> request. • If HTTP 200 OK: ◦ Deserialize response → <code>List<c3Header></code> ◦ Filter only <code>payerType == "ER"</code> If API fails: • Reads error message

			• Returns failure response
			5. Loop Through Each C3 Header For every C3 submission header: 5.1 Validate Header Status Process only when: • <code>receivedBy == "c3svc"</code> • OR (<code>receivedBy != "c3svc"</code> AND <code>c3Status == "VAC"</code>) Determines: • <code>islocked = 1</code> if VAC • Otherwise <code>islocked = 0</code>
			6. Check If C3 Already Imported • Extracts: ◦ <code>paymonth</code> ◦ <code>payyear</code> ◦ <code>sequenceNo</code> • Calls stored procedure: • <code>check_C3CreatedImport</code> • If matching schedule number already exists: ◦ Skip this C3

			◦ Increment count
			7. Fetch Detailed C3 Submission (If Not Imported) • Builds second API URL: • /C3/{payerId}/C3Submitted/{month},{year},{sequenceNo},ER,EE • Calls API again with authentication • If HTTP 200 OK: ◦ Deserialize into submitC3string
			8. Process Header & Wage Data Only proceeds if: • ipWages.Count > 0 • OR nilReturn == "1" 8.1 Calculate Totals • Loops through wages: ◦ Sums employer levy ◦ Sums total wages
			9. Insert C3 Header into

			Database • Inserts into: • PROCESS_C3Header • Includes: ◦ Period ◦ Totals ◦ Employer ID ◦ Schedule number ◦ Lock status ◦ Submitted date • Retrieves generated C3HEADERID.
			10. Insert Wage Details For each ipWages record: • Determine pay frequency (W, E2W, M, 2M) • Insert into: • Process_Contributions • Includes: ◦ SSN ◦ Wages ◦ Social Security ◦ Levy amounts ◦ Employment dates
			11. Tracking Missing or Existing

			Data • If no wages and not nil return: ◦ Adds period & schedule to detailsNotfound • If all records already exist: ◦ Returns: ◦ "Data for this month and year already exists."
			12. Final Response Logic Success cases: • Some data imported: • "Submitted C3 Imported Successfully" • Some periods missing: • "Data Not Found For This Period and Schedule..." Failure cases: • API failure • Duplicate data • Exception
			13. Exception Handling • Logs: ◦ Controller name ◦ Action name

			○ Exception message & stack trace • Returns HTTP 500 with error message
			14. Final Return • Returns: ○ 200 OK for success ○ 400 / 500 for failure
Import C3 File	POST <code>/api/C3/UploadC3File2</code> POST <code>/api/C3/SaveNContinue</code> POST <code>/api/C3/EditC3EmployeeListing</code>	MasterHolidayPayDetails MasterEmployee Mastercomapany process_contributions PROCESS_C3Header BonusPayDetails	1. Method Entry & Exception Handling • Method is an async API action. • Entire logic is wrapped in <code>try-catch</code>. • Any unhandled exception: ○ Logged via <code>LoggingHelper</code> ○ Returns HTTP 500
			2. File Presence & Extension Validation 2.1 File Exists Check

			<pre>if (obj.file == null obj.file.Length == 0)</pre> - Rejects request if no file uploaded. 2.2 File Extension Check <code>Path.GetExtension(obj.file.FileName)</code> - Only allows .c3 files. - Any other extension → BadRequest
			3. File Storage Setup 3.1 Build Storage Path <code>wwwroot/ImportC3File</code> 3.2 Ensure Directory Exists - Creates directory if missing. 3.3 Generate Unique File Name - Uses <code>Guid.NewGuid()</code> to avoid overwrite. - Preserves original .c3 extension. 3.4 Save File to Disk - Uses <code>FileStream</code>

			Writes uploaded file asynchronously.
			4. Read File Content - Reads all lines using: <code>File.ReadAllLines(reportPath)</code> - Initializes: <code>C3FileData c3Data</code> <code>lineNumber</code> for error tracking
			5. Line-by-Line File Validation Each line is processed sequentially. 5.1 Common Validation - Blank line → BadRequest - CSV split by comma: <pre>line.Split(',').Select(p => p.Trim())</pre>
			6. HDR Line Processing (<code>parts[0] == "HDR"</code>) Required Validations

			• Must have 5 fields • Registration number length must be 6 • Company must exist in DB • RegNumber in file must match DB • Company name must match DB (after cleanup) • Date must be in dd/MM/yyyy Extracted Values • RegNo • Period Month • Period Year Stored In c3Data.Header = new ProcessC3header
			7. FTR Line Processing (parts[0] == "FTR") Required Validations • Must have 8 fields • RegNumber length = 6 • Date format = dd/MM/yyyy • Columns 4-8 must: ○ Not be empty ○ Be valid decimals

			If any validation fails → BadRequest
			8. Detail Record Processing (Employee Records) Triggered when line is not HDR or FTR Structural Validation - Must contain 24 columns - SSN must be 6 characters Date Validation - Start Date (<code>parts[5]</code>) → optional, but valid format if present - End Date (<code>parts[6]</code>) → optional, but valid format if present Numeric Validation Columns 8–13 - Must be integers Columns 14–24 - Must be decimal values

			Any failure → BadRequest with line number
			9. Parse & Insert C3 Data After all lines pass validation: <pre>parsedData = await _C3.ParseC3File2(reportPath, companyId, userId)</pre> If Parsing Succeeds - Returns: <pre>{ "status": true, "message": "C3 file processed successfully", "data": parsedData.Data }</pre> If Parsing Fails - Returns: <pre>{ "status": false, "message": parsedData.Message, "data": parsedData.Data }</pre>
			10. Exception Handling

			If any unexpected error occurs: • Logs controller & action • Returns:
NW Director	GET /api/NonWorkingDirector/NonWorkingDirector_List GET /api/NonWorkingDirector/NonWorkingDirectorGetById POST /api/NonWorkingDirector/Save_Director	MasterEmployee Mastercomapany process_contributions PROCESS_C3Header MasterEmployeeIncomes	1. Method Entry & Response Initialization • Method is an async controller action (though it uses sync DB calls). • Creates a ResponseModel object to store: ◦ Status ◦ Statuscode ◦ Message ◦ Data 2. Database Connection Setup • Retrieves a static SQL connection: • C3WizardLayerConn_BaseData.StaticSqlConnection • Sets audit session context.

			• Opens the connection if it is closed.
			3. Fetch Directors Already Used in C3 Purpose To identify non-working directors that already have C3 contributions created, so they cannot be deleted. SQL Query Executed <pre>SELECT DISTINCT PC.SSN FROM Process_Contributions PC INNER JOIN PROCESS_C3Header PH ON PC.C3HEADERID = PH.C3HEADERID WHERE PH.ForDirector = 1 AND PH.EmployerID = @CompanyId</pre> <i>(Note: current code concatenates CompanyId directly)</i> Result • Stored in dtC3C • Contains SSNs that are already linked to C3 records
			4. Fetch Non-Working Directors Master List

			• Calls business layer: BLMasterEmployee.MasterNWDirectorCollection_grid(CompanyId) • Returns a list of non-working directors for the company
			5. Build View Model List For each employee in the master list: 5.1 Create New Object • Creates a new BLMasterEmployee object (blemp) • Copies required fields: ○ SSN ○ EmployeeID ○ Name ○ Appointment & termination dates ○ Address, department, designation ○ Pay period 5.2 Format Dates • Converts dates to display format

			using Helper.DateFormat
			6. Check C3 Dependency (Delete Protection) Logic - Checks if the employee SSN exists in dtC3C <pre>DataRow[] DRC3C = dtC3C.Select("SSN = " + btemp.EmployeeID);</pre> If Found - Employee already used in C3 - Flags set: <pre>btemp.IsC3Created = true; btemp.Candelete = false;</pre> This prevents deletion in UI.
			7. Add to View List Adds processed employee object to: Viewemployeelist

			8. Return API Response Success Case If list is not null: <pre>{ "Status": true, "Statuscode": 200, "Message": "Data retrieved successfully!", "Data": Viewemployeeelist }</pre> No Data Case <pre>{ "Status": true, "Statuscode": 400, "Message": "No data found!", "Data": [] }</pre>
			9. Exception Handling • Logs: ○ Controller name ○ Action name ○ Error message & stack trace • Rethrows exception (throw ex;)

NW Director Payroll	GET /api/NonDirector/NewDirectorMainDetails POST /api/NonDirector/NewDirectorDetailsC3 GET /api/NonDirector/NwCheckC3Created POST /api/NonDirector/NWSaveNContinue	MasterEmployee Mastercomapany process_contributions PROCESS_C3Header Deductions_Tax_Table_Details	1. Method Entry - This is an async controller action. - Entire logic is wrapped in a try-catch block to handle runtime errors safely.
			2. Load Default Director Data var data = await _NonDirectorRepo.load_defaults(C ompanyId); - Calls repository method to fetch default C3 header data for the given company. - Expected to return a collection of entities (C3 headers related to directors).
			3. Load Pay Period Dropdown Data var periodList = _NonDirectorRepo.PayPeriodDropDo wnData(CompanyId); Retrieves pay-period values (e.g., Monthly, Weekly, etc.)

			Used by UI dropdowns.
			4. Map Data to View Model <pre>var data1 = data.Select(e => new C3HeaderVM { ... }).ToList();</pre> Mapping Includes: - Header identifiers - Registration number - Period month & year - Total wages & contributions - Levy penalties - Insert date - Finalization & submission status - Schedule number - Nil-return flag 👉 This step transforms DB entities into a UI-friendly ViewModel.
			5. Success Response <pre>if (data != null)</pre> - If data exists: - Returns HTTP 200 (Ok) - Status = true - Includes: - data1 (C3 header list) - periodList

			(dropdown values) Response Structure <pre>{ "Status": true, "Message": "data found..!", "Data": { "data1": [...], "periodList": [...] } }</pre> 1. Method Entry • HTTP POST API endpoint: NewDirectorDetailsC3 • Parameters received: ◦ monthNo → Month number ◦ Year → Year (string) ◦ CompanyId ◦ CmbPayPeriod → Selected pay periods ◦ isNilReturn → Whether this is a Nil return C3 • Entire method is wrapped in try-catch for error safety.
			2. Load Pay Period Master Data

			<pre>var periodList = await _NonDirectorRepo.PayPeriodDropDo wnData (CompanyId) ;</pre> Fetches available pay periods for UI dropdown usage.
			3. Load Director C3 Contribution Data Conditional Logic <pre>var data = isNilReturn ? new List<C3Contributions>() : await _NonDirectorRepo.LoadDirectorC3(monthNo, Year, CompanyId, CmbPayPeriod, 0) ;</pre> If Nil Return: - Skip DB fetch - Initialize empty contribution list Else: - Load director contribution data for the selected: - Month - Year - Company - Pay periods

			4. Normalize Pay Period Display Text For each record in data: Converts pay period codes into user-friendly labels: <table><tr><th>Code</th><th>Display Text</th></tr><tr><td>W</td><td>W - Weekly</td></tr><tr><td>M</td><td>M - Monthly</td></tr><tr><td>E2W</td><td>E2W - Every Two Weeks</td></tr><tr><td>2M</td><td>2M - Twice Monthly</td></tr></table> This is strictly for UI display.	Code	Display Text	W	W - Weekly	M	M - Monthly	E2W	E2W - Every Two Weeks	2M	2M - Twice Monthly
Code	Display Text												
W	W - Weekly												
M	M - Monthly												
E2W	E2W - Every Two Weeks												
2M	2M - Twice Monthly												
			5. Validate Data Availability if (data != null && data.Count != 0)										
			6. Check Existing Director C3 Submission Purpose										

			• To prevent duplicate submissions • To decide next schedule number Query <pre>var isC3Created = _context.ProcessC3headers .Where(e => e.PeriodMonth == monthNo.ToString() && e.PeriodYear == Year && e.EmployerId == CompanyId && e.ForDirector == true) .OrderByDescending(e => e.ScheduleNo) .FirstOrDefault();</pre>
			7. Build User Message If: • C3 exists and • It has already been submitted Then: "C3 has already been submitted for this month. Do you want to create a new C3 with another schedule X?" Where:

			X = ScheduleNo + 1
			8. Success Response <pre>{ "Status": true, "Message": "data found..!", "Msg": "optional warning message", "Data": { "isNilReturn": true/false, "data": [...], "periodList": [...] } }</pre> • Returned with HTTP 200 OK
			9. No Data Found Case HTTP 400 <pre>{ "Status": false, "Message": "data Not found..!", "Data": "" }</pre>

Nw Director Reports

GET
`/api/NonDirector/NWDirectorReports`

GET
`/api/NonWorkingDirector/DownloadSubmittedC3_Click`

GET
`/api/NonWorkingDirector/ImportLastC3_Click`

MasterEmployee
Mastercomapany
process_contributions
PROCESS_C3Header
Deductions_Tax_Table_Details

1. Method Entry

- Async controller action used to **generate Non-Working Director (NWD) C3 reports**
- Accepts:
 - Month range (MonthF, MonthTO)
 - Year range (Year, endYear)
 - CompanyId
- Wrapped in a **try-catch** for safe error handling.

2. Normalize Month Range Inputs

The method converts loose user input into **validated filter boundaries**.

Month Rules Applied

Case	R
From month only	StartMonths = Month
From + To	StartMonths = Month
To month only	StartMonths = null,

			NeitherBoth null										
			3. Normalize Year Range Inputs Same logic applied to years: <table><tr><th>Case</th><th>Result</th></tr><tr><td>Start year only</td><td>StartYear = Year, EndY</td></tr><tr><td>Start + End</td><td>Both assigned</td></tr><tr><td>End year only</td><td>StartYear = null, EndY</td></tr><tr><td>Neither</td><td>Both null</td></tr></table>	Case	Result	Start year only	StartYear = Year, EndY	Start + End	Both assigned	End year only	StartYear = null, EndY	Neither	Both null
Case	Result												
Start year only	StartYear = Year, EndY												
Start + End	Both assigned												
End year only	StartYear = null, EndY												
Neither	Both null												
			4. Fetch Director C3 Data from Database DB Call - Opens static SQL connection - Executes stored procedure: Get_Director_C3_Genrated_data (@CompanyId) Output - Fills a DataTable (dt) with all										

			director C3 headers for the company.
			5. Build Director C3 Header List For each row in dt: 5.1 Submission Filter Only process rows where: Is_Fianalize == true OR Is_submitted == true
			5.2 Map DB Row → C3Header Fields populated include: • Header ID, Reg No • Period month/year (numeric + text) • Total wages & contributions • Levy amounts & penalties • Created date • Finalize & submit flags • Schedule number • Notes & Nil-return flag • Import source (BEMA)
			5.3 Submission Status UI Fields

			• Text (C3 Submitted / Not Submitted) • Color (Black / Red) • Icon path
			5.4 Payment Information Lookup From <code>OnlinePayments</code> : • Checks if payment exists (<code>AUTHORIZED</code> or <code>Offline Payment</code>) • Extracts: ◦ Payment status ◦ Transaction ID ◦ Mode ◦ Receipt details
			5.5 Add to Result List Each valid <code>C3Header</code> is added to: <code>listDVOC3Header</code>
			6. Shape Report Output Maps <code>listDVOC3Header</code> into a lightweight report object :

			Includes: • Year / Month • Wages & levy • Schedule number • Paid status • Payment amount • Nil return flag • Payment transaction details Stored in: repotList
			7. Build Date Filters Start Date • Uses: ◦ StartYear + StartMonths ◦ Defaults to January if month missing End Date • Uses: ◦ EndYear + endMonths ◦ Defaults to December if month missing
			8. Apply Date Filtering

			Filters repotList by comparing: <pre>new DateTime(year, month, 1)</pre> Ensures: <pre>startDate ≤ periodDate ≤ endDate</pre> Only applied if any date filter is provided.
			9. Return Response Data Found <pre>{ "Status": true, "Message": "data found", "Data": repotList }</pre> No Records <pre>{ "Status": true, "Message": "data found", "Data": null }</pre> ===== ===== =====  ImportLastC3_Click - Code Flow

			1. Method Entry & Preconditions Purpose: Import the last submitted Non-Working Director (NW) C3 from the external BEMA/C3 service and save it locally. Initial checks <pre>if (ConnectionInfom.HasConnection() && ConnectionInfom.APIConnection(_c onfiguration))</pre> Ensures: Internet connectivity API configuration is valid ✗ If failed → returns <i>BadRequest</i> with connection error.
			2. Setup & Configuration Fetch employer registration number: <pre>string regn = regno(CompanyId);</pre> Build service URL to fetch last submitted C3:

			/C3/{RegNo}/C3Submitted/ER/1,NW • Create <code>HttpClient</code> ◦ Timeout: 2 minutes ◦ Content type: JSON ◦ Authorization: ▪ Uses BEMA credentials if enabled ▪ Otherwise uses local LoginId + Password
			3. Fetch Last Submitted C3 Header <code>HttpResponseMessage response = httpClient.GetAsync (NewsURL) .Result;</code> If HTTP 200 OK • Deserialize response into: <code>c3Header C3Header_model</code> • Continue only if <code>payerType == "ER"</code>
			4. Extract Period Information From <code>C3Header_model.period:</code>

			• Parse Month • Parse Year						
			5. Check if Director C3 Already Imported Stored procedure: <pre>check_Director_C3Created_Import</pre> Inputs • Month • Year • CompanyId Result <table><thead><tr><th>Condition</th><th>Action</th></tr></thead><tbody><tr><td>Data exists</td><td> Return 400 – “Data already</td></tr><tr><td>No data</td><td> Continue import</td></tr></tbody></table>	Condition	Action	Data exists	 Return 400 – “Data already	No data	 Continue import
Condition	Action								
Data exists	 Return 400 – “Data already								
No data	 Continue import								
			6. Fetch Full Submitted C3 Details Build second API URL: <pre>/C3/{payerId}/C3Submitted/{month}, {year}, {sequenceNo}, {payerType</pre>						

			<pre>},NW</pre> • Fetch: <pre>submitC3string submitC3_model</pre> Includes: • c3Header • nonWorkingDirectorWages
			7. Validate C3 Status C3 is importable only if: <pre>receivedBy == "c3svc" OR (receivedBy != "c3svc" && c3Status == "VAC")</pre> • Determine lock status: <pre>islocked = (receivedBy != "c3svc" && c3Status == "VAC") ? 1 : 0;</pre>
			8. Calculate Director Totals • Calculate: <pre>totalwagesDir = sum(nonWorkingDirectorWages.wage s)</pre>

			Determine if month has 5 Mondays <pre>IsWeekfifth = MondaysInMonth(month, year) == 5</pre>
			9. Insert C3 Header (PROCESS_C3Header) Key fields inserted - Period Month & Year - Total wages - Social security, levy, penalties - Employer ID - Schedule number - Director flag - Submission & finalize flags - Imported from BEMA = true - Submitted date ➡ After insert: - Fetch generated C3HEADERID
			10. Insert Director Wage Records For each Non-Working Director :

			Resolve Employee ID - Match SSN with: BLMasterEmployee.MasterNWDirectorCollection_grid(CompanyId) Insert into Process_Contributions - Assign wages: - Week 4 / Week 5 logic based on IsWeekfifth - Set: - Director wage - Levy amount - Start / End dates
			11. Successful Completion If all inserts succeed: { "Status": true, "Message": "Submitted Last C3 Imported Successfully", "StatusCode": 200 }
			12. API Error Handling If API returns non-200: Deserialize Apistatus

			Return API message as BadRequest
Payment Details	POST <code>/api/Payment/AdminTranactionHistory</code>	MasterEmployee Mastercomapany process_contributions PROCESS_C3Header OnlinePayments	 getAllCompanyTrans - Code Flow Explanation  Purpose Fetch transaction history (C3 payments) for: Company Director Self-Employed SSB user with: - Filters (status, date range, company, user, role type)

			• Pagination
			1 Method Inputs PaymentStatus, FromDate, ToDate, CompanyId, UserId, types, pageNumber, pageSize types controls role-based behavior: • Company • SSB • SelfEmployee
			2 Initial Setup Creates: • ResponseModel • Multiple lists to separate data: ◦ listDVOC3Header → Company C3 ◦ Director_listDVOC3Header → Director C3 ◦ selfList → Self-employed C3
			3 Database Call Stored Procedure

			Get_TransactionHistory Parameters passed <table><tr><th>Parameter</th><th>Source</th></tr><tr><td>PaymentStatus</td><td>Filter</td></tr><tr><td>FromDate</td><td>Filter</td></tr><tr><td>ToDate</td><td>Filter</td></tr><tr><td>CompanyId</td><td>Filter</td></tr><tr><td>UserId</td><td>Filter</td></tr><tr><td>types</td><td>Role-based logic</td></tr></table> ➡ Data is returned as a DataSet with multiple tables	Parameter	Source	PaymentStatus	Filter	FromDate	Filter	ToDate	Filter	CompanyId	Filter	UserId	Filter	types	Role-based logic
Parameter	Source																
PaymentStatus	Filter																
FromDate	Filter																
ToDate	Filter																
CompanyId	Filter																
UserId	Filter																
types	Role-based logic																
			4 Loop Through Result Tables for (int i = 0; i < ds.Tables.Count; i++) Each table represents different role-based data.														
			5 Row-Level Role Routing Logic														

			A. Company / Director (SSB or Company role) <pre>if ((types == SSB && i == 0) types == Company)</pre> ➤ If row["Types"] == "company" ➡ Company C3 record Mapped into <code>listDVOC3Header</code> Fields mapped: • HeaderID • RegNo • Period (Month & Year) • Wages • Contributions • Penalties • PaymentHistory (JSON → <code>PayDetails</code>) • Schedule No • UserId
			➤ Else → Director C3 record ➡ Stored in <code>Director_listDVOC3Header</code> Same mapping logic but stored separately.

			B. Self-Employed or SSB (Self C3) else if (types == SelfEmployee types == SSB) ➡ Maps Self-employed C3: - SEC3ID - Period - Total wages - Contributions - Penalties - Payment history Stored in selfList.
			6 Normalize Data (Projection) Each list is re-mapped to C3HeaderVM to: - Normalize fields - Compute totals - Remove unused columns Lists created: - DVOC3Header → Company - Director → Director - Selfdata → Self-employed

			7 Merge All Data DVOC3Header.AddRange(Director); DVOC3Header.AddRange(selfList); ➡ Final unified transaction list.
			8 Pagination Logic totalRecords = DVOC3Header.Count totalPages = ceil(totalRecords / pageSize) pagedDir = DVOC3Header .Skip(pageNumber * pageSize) .Take(pageSize) ⚠ Important • pageNumber is zero-based • Page 1 = pageNumber = 0
			9 Response Construction ✓ If records exist { "Status": true, "Message": "Data retrieved successfully!", "Data": { "TransactionList": { "TotalRecords": X, "PageNumber": N,

			<pre>"PageSize": Y, "TotalPages": Z, "Records": [...] } } }</pre>
User Management	GET <code>/api/Administration/LoadUsers</code>	MasterEmployee Mastercomapany secusersprofile SECUsers	1 Method Entry GetEditUserManagement(int UserId) • UserId → ID of the user to edit
	GET <code>/api/Administration/GetEditUserManagement</code>		
	POST <code>/api/Administration/SaveUser/UpdateUser</code>		2 Load All Users blList_ALL = BLSECUsers.SECUsersCollection(). ToList(); • Loads all SEC users into memory • ⚠ Inefficient if user table is large
			3 Filter Requested User blList = blList_ALL.Where(t => t.UserId == UserId).ToList();

			• Extracts only the requested user
			4 Validate User Exists <pre>if (blList.Count == 1)</pre> • Proceeds only if exactly one user is found •  Otherwise → returns <code>Ok(Userid)</code> (not ideal API behavior)
			5 Map User Data to BLSECUsers Creates a new data object and maps properties safely: Mapped Fields • UserId • FirstName • MiddleName • LastName • LoginId • EmailId • RoleId • EmpId • Status (Active / Inactive) • ParentUserId

			• IsPPOC ✓ Handles nulls and empty strings
			6 Build Company Dropdown List Using EF Core join: <pre>MasterCompanies JOIN SECUsersProfiles</pre> Conditions • Company is active • Profile is active • Parent user logic: ◦ If parentuserid == 0 → use Userid ◦ Else → use parentuserid Dropdown Fields • Company ID • Company logo URL • Registration number • Company name • Seleted_Comp flag (already assigned to user)

			7 Role-Based Shortcut (RoleId == 2) if (data.RoleId == 2) Immediately returns: { Message, Data, dropdown } ➡ No profile image handling
			8 Profile Image Handling Inside try block: 1. Read image name from DB 2. Build public image URL: {Scheme}:{Host}/UserImage/{ImageName} 3. Assign to: data.ProfileImage
			9 Final Success Response { Message: "Data get successfully",

			<pre>Data: userData, dropdown: companyList }</pre> Error Handling - Any exception: - Logged using LoggingHelper - Method continues silently (no error response returned) ===== ===== ===== SaveUserandUpdateUser – Code Flow Purpose This API handles both: Create new user (mode == 1) Update existing user (mode == 2) It also: - Uploads profile image
--	--	--	--

			• Encrypts password • Assigns companies (SECUsersProfiles)
			1 Method Entry SaveUserandUpdateUser (UserViewModel userViewModel) • Data comes as multipart/form-data • Supports file upload (UserImage)
			2 Null Validation if (userViewModel == null) ✗ If null → returns 400 Bad Request
			3 Deserialize Company Mapping (Optional) SECUsersProfilesString → List<SECUsersProfileVm> • Converts JSON string to object list • Used for company assignment

			4 Environment Validation if (_environment == null)  If missing → 500 Server Error
			5 Default Password & Image Setup - Default password: Admin@123 - Image filename initialized as null
			6 Build User Object (BLSECUsers) secuser = new BLSECUsers { ... } Key Fields - Name, LoginId, Email - Password (encrypted) - RoleId - CompanyId - Parent User - IsActive = true  If password not provided → default password used

			7 Profile Image Upload (Optional) If <code>UserImage</code> exists: - Save file to: <code>wwwroot/UserImage/</code> - Generate GUID filename - Assign to: <code>secuser.ProfileImage</code> ✗ Any failure → 500 error
			↺ MAIN LOGIC (MODE BASED)
			● MODE = 1 → CREATE USER 8 Check Duplicate LoginId <pre>_context.Secusers.Where(LoginId)</pre> ✗ If exists → 409 Conflict
			9 Save New User <pre>secuser.SaveNew();</pre> Sets audit fields

			• Saves user record
			 Token Generation After saving: <code>token = Guid.NewGuid() userstts = "A"</code>
			 Company Assignment If <code>SECUsersProfiles</code> exists: • Loop through companies • Insert missing mappings If not: • Assign default company
			 Save Changes <code>_context.SaveChanges();</code>
			 Success Response <code>{ Status: true, Message: "User saved successfully" }</code>

			 MODE = 2 → UPDATE USER
			 Check Duplicate Username LoginId exists && UserId != current  If found → 409 Conflict
			 Fetch Existing User LoginId + UserId  If not found → User data not exists
			 Update User Details • Name • Email (if changed) • Parent user • Audit fields • Profile image (if uploaded) <code>secuser.Update () ;</code>
			 Update Company Assignments For each company:

			• If exists → update IsActive • If not → insert new mapping
			 Save Changes <code>_context.SaveChanges();</code>
			 Build Image URL <code>https://host/UserImage/{filename}</code>
			 Success Response { Status: true, Message: "User update successfully" }
			 Invalid Mode If mode ≠ 1 or 2: 400 - Invalid mode
			 Global Exception Handling • Logs error using LoggingHelper • Returns 500 Internal Server Error

Logged In History	GET <code>/api/Administration/LoggedInHistory</code>	LoginLog SECUsers	This API retrieves the login history of users with: • Pagination • Optional date filtering • Role/Company-based filtering (via <code>companyId</code> and <code>isSelfEmployed</code>)
			1 Method Entry <code>LoggedInHistory(int companyId, bool isSelfEmployed, int RoleId, int pageNumber, int pageSize, DateTime? fromDate, DateTime? toDate)</code> • Inputs: ○ <code>companyId</code>: Company filter ○ <code>isSelfEmployed</code>: Filter for self-employed users ○ <code>RoleId</code>: User role (currently not used in filtering) ○ <code>pageNumber</code> & <code>pageSize</code>: Pagination ○ <code>fromDate</code> & <code>toDate</code>:

			Optional date filter
			2 Pagination Validation if (pageNumber < 0) pageNumber = 0; if (pageSize < 1) pageSize = 10; - Ensures: - Page number ≥ 0 - Page size ≥ 1 (default 10) ✗ Note: pageNumber here is 0-based, not 1-based.
			3 Fetch All Login Logs List<BLLoginLog> loginList = BLLoginLog.LoginCollection(companyId, isSelfEmployed).ToList(); - Fetches all login logs for the company and user type - Currently loads all data in memory (potential performance issue for large datasets)
			4 Apply fromDate Filter if (fromDate != null) loginList =

			<pre>loginList.Where(e => e.LoginTime >= fromDate.Value).ToList();</pre> • Keeps records with LoginTime >= fromDate • Converts filtered results back to list
			5 Apply toDate Filter <pre>if (toDate != null) { toDate = toDate.Value.AddDays(1); loginList = loginList.Where(e => e.LoginTime <= toDate.Value).ToList(); }</pre> • Includes records up to the end of toDate day • Again converts filtered results back to list
			6 Pagination <pre>var totalRecords = loginList.Count(); var newList = loginList.Skip(pageNumber * pageSize).Take(pageSize);</pre> • Calculates total records • Applies Skip and Take to get current page

			• Note: pageNumber starts from 0
			7 Check if any records exist if (loginList.Any()) •  If records exist → returns 200 OK •  If empty → returns 404 Not Found
			8 Build Response new ResponseModel { Status = true, Message = "Login logs retrieved successfully.", Data = new { TotalRecords = totalRecords, PageNumber = pageNumber, PageSize = pageSize, TotalPages = (int)Math.Ceiling((double)totalR ecords / pageSize), Records = newList }, Statuscode = 200 } • Includes: ○ TotalRecords ○ PageNumber ○ PageSize

			- o TotalPages - o Records (current page)
			9 Exception Handling catch (Exception ex) { LoggingHelper.LogError(...); return StatusCode(500, ...); } • Logs error • Returns 500 Internal Server Error if any unhandled exception occurs
Contact us	POST <code>/api/Administration/SaveContactUs</code>	ContactusLogs Mastercomapany	1 Method Entry SaveContactUs ([FromBody] ContactUsRequest request) • Input: request object with: - o CompanyId - o UserId - o Description - o RegNo (optional, used in email/log)
			2 Determine recipient email var Email = ""; var IsTestMail = _configuration["IsTestMail"];

			<pre>if (IsTestMail == "1") Email = _configuration["TestMail"];</pre> • If IsTestMail is "1" → send all emails to test email • Otherwise, Email will be set later from the company database.
			3 Validate request <pre>if (request == null) return BadRequest("Invalid request");</pre> • If request is null → 400 Bad Request is returned.
			4 Fetch company details from database <pre>SqlConnection staticConnection = C3WizardLayerConn_BaseData.Stati cSqlConnection; C3WizardLayerConn_BaseData.SetAu ditSessionContextAsync(staticCon nection); SqlCommand CurrentCompany = new SqlCommand("SELECT mc.Company_Id, mc.REG_NUMBER, mc.Company_Name, mc.Email, mc.Contact_Person, mc.Mobile, mc.Landline " +</pre>

			<pre>"FROM MasterCompany mc WHERE mc.Isactive = 1 AND mc.Company_Id=" + request.CompanyId, staticConnection); if (staticConnection.State == ConnectionState.Closed) staticConnection.Open(); SqlDataReader dr = CurrentCompany.ExecuteReader();</pre> • Opens a static SQL connection • Executes query to fetch the company info • Reads into: ○ RegNo ○ ContactPerson ○ CompanyName ○ EmailId ○ Contectno (Mobile + Landline)  Security risk: Using string concatenation for SQL query → SQL Injection vulnerability. Use parameterized queries instead.
			5 Close SQL connection <pre>staticConnection.Close();</pre> • Closes the SQL connection after reading the data.

			6 Send Email <pre>bool emailResult = await SendEmailAsync(Email, // recipient email Helper.ReplaceCharacter(CompanyName), request.Description, // message body ContactPerson, Contectno, RegNo);</pre> • Calls a service to send email. • Returns <code>true</code> if email is sent successfully, <code>false</code> otherwise.
			7 Log Contact Us in database (if email sent) <pre>if (emailResult) { var contactUsLog = new ContactUsLog { UserId = request.UserId, CompanyId = request.CompanyId, RegistrationNo = Convert.ToInt32(request.RegNo), Emailid = Email, Description = request.Description,</pre>

			<pre>InsertDate = DateTime.Now }; _context.ContactUsLogs.Add(conta ctUsLog); await _context.SaveChangesAsync(); }</pre> • Creates a new record in ContactUsLogs • Saves it asynchronously to the database
			8 Return success or failure response •  If email sent and logged successfully: <pre>return Ok(new { Status = true, Message = "The message has been sent successfully.", Statuscode = 200 });</pre> •  If email failed: <pre>return Ok(new { Status = false, Message = "Mail sending failed.", Statuscode = 400 });</pre>
			9 Exception Handling

			<pre>catch (Exception ex) { LoggingHelper.LogError(...); return Ok(new { Status = false, Message = "Mail sending failed.", Statuscode = 400 }); }</pre> • Logs any unexpected errors • Returns failure response • Ensures that the API never throws unhandled exceptions
--	--	--	--